

Search Typeahead System — Project Report

HLD101 Assignment · Author: **Parth Dagia** · Repository: github.com/parthdagia05/HLS-ASSIGN

A backend-focused search typeahead (autocomplete) system, similar to the suggestion box in Google / Amazon / YouTube. It suggests popular queries as the user types, records searches, serves suggestions from a **distributed cache** (Redis nodes routed with **consistent hashing**), ranks results with a **recency-aware "trending"** score, and absorbs write pressure with **batched writes**.

2.65 ms

suggestion latency (server p95,
warm)

96%

cache hit rate

≈50×

write reduction via batching

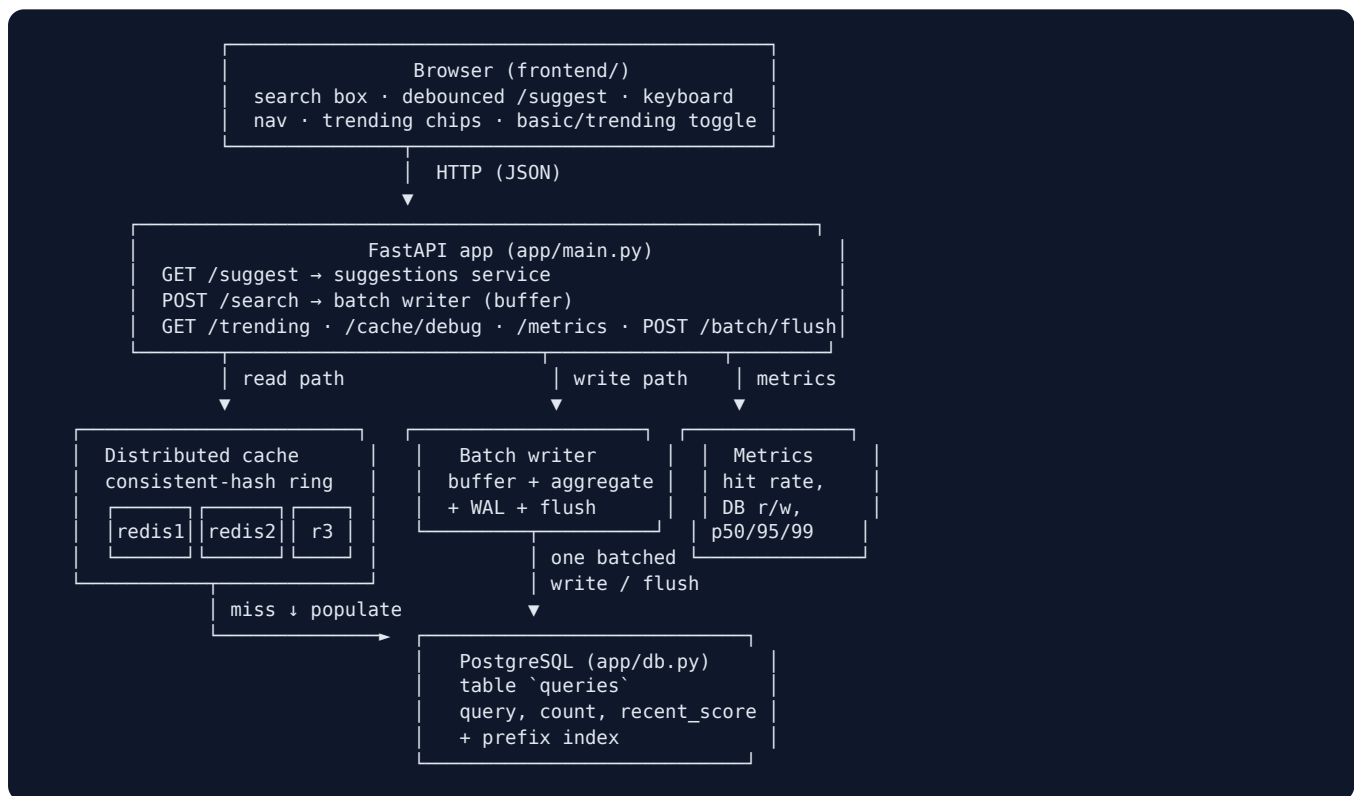
~1/N

keys remapped on node loss

Tech stack: Python 3.12 + FastAPI · PostgreSQL 16 (primary store) · Redis (3 logical cache nodes, app-level consistent hashing) · plain HTML/CSS/JS frontend · Docker Compose for local infrastructure.

1. Architecture

Two independent paths. The **read path** (suggestions) is optimised for latency — cache first, database only on a miss. The **write path** (search submissions) is optimised for throughput — buffer in memory, aggregate, and write to the database in batches.



Data model

A single table keeps the system easy to reason about. The index `idx_queries_prefix ON queries (query text_pattern_ops)` makes `WHERE query LIKE 'prefix%'` use an index even on a UTF-8 database, which keeps prefix lookups fast at 120k rows.

Column	Meaning
<code>query</code> (PK)	search string, stored normalised (lowercased, trimmed)
<code>count</code>	all-time number of searches — historical popularity
<code>recent_score</code>	time-decayed recent activity — drives trending
<code>recent_score_updated_at</code>	when <code>recent_score</code> was last refreshed (decay anchor)
<code>last_searched_at</code>	last submission time (diagnostics)

Concurrency model

Intentionally synchronous and simple: FastAPI runs sync endpoints in a threadpool; PostgreSQL access goes through a connection pool; the batch writer runs on its own daemon thread with lock-guarded shared state. No asyncio, so there are no `await` subtleties to explain.

2. Dataset — source & loading

The system needs `query, count` rows ($\geq 100,000$ unique queries).

- **Default (synthetic, reproducible):** `scripts/generate_dataset.py` builds **120,000** unique queries from real-world head terms ("iphone", "java"...) + modifiers ("review", "near me", "2024"...), with a **Zipf-like (long-tail)** count distribution — the shape real search traffic has. It is deterministic (fixed seed), so the dataset is identical on every run and needs no download. Output: `data/queries.csv`.
- **Using a real dataset instead:** produce a CSV with a `query, count` header (any open dataset of search queries / product names / page titles; aggregate to derive counts if absent) and run `python scripts/load_dataset.py --csv path/to/your.csv`.

Loading: `load_dataset.py` bulk-loads via PostgreSQL `COPY` into a staging temp table, then does a single `INSERT ... ON CONFLICT DO UPDATE` into `queries` — fast, and idempotent (safe to re-run). Every query is normalised on load to match how the API normalises the user's prefix.

```
python scripts/generate_dataset.py # writes data/queries.csv (120k rows)
python scripts/load_dataset.py     # COPY → staging → upsert into queries
```

3. API documentation

Endpoint	Method	Purpose / behaviour
<code>/suggest?q=<prefix>&mode=</code>	GET	Up to 10 prefix matches. <code>mode = basic</code> (all-time count) or <code>trending</code> (recency-aware). Empty/no-match → empty list, never an error.
<code>/search</code>	POST	Body <code>{"query": "..."} → {"message": "Searched"}</code> ; records the query via the batch writer (no synchronous DB write).
<code>/trending?limit=</code>	GET	Currently trending searches (decayed recency, count as tie-breaker).
<code>/cache/debug?prefix=</code>	GET	Which cache node owns the prefix + whether it is a hit or a miss.
<code>/metrics</code>	GET	Cache hit rate, DB read/write counts, suggestion latency p50/p95/p99, per-node health.
<code>/batch/flush</code>	POST	Force a batch flush (for demos/tests).
<code>/health</code>	GET	Liveness + number of queries loaded.

Interactive, auto-generated docs are also served at `/docs` (OpenAPI/Swagger).

```
curl "localhost:8000/suggest?q=iph&mode=trending"
→ {"prefix":"iph","mode":"trending","suggestions":[{"query":"iphone","count":5418280}, ...]}

curl -X POST localhost:8000/search -H 'Content-Type: application/json' -d '{"query":"iphone 15"}'
→ {"message":"Searched","query":"iphone 15"}

curl "localhost:8000/cache/debug?prefix=iph"
→ {"node":"localhost:6391","status":"hit","cache_key":"suggest:trending:iph", ...}
```

4. Design choices & trade-offs

Why PostgreSQL + an indexed prefix scan (not a trie / search engine)?

The dataset (100k–1M rows) is small enough that an indexed `LIKE 'prefix%'` returns the top 10 in single-digit milliseconds, and a relational table makes counts, recency and batched upserts trivial and durable. The cache absorbs the read volume, so the database is rarely the bottleneck. A trie or Elasticsearch would be the next step at much larger scale.

Why our own consistent hashing (not Redis Cluster)?

With plain `hash(key) % N`, adding/removing a node changes the bucket for almost every key, dumping the entire load onto the database at once. **Consistent hashing** places nodes and keys on a circle; a key belongs to the next node clockwise, so a node change only moves $\sim 1/N$ of keys. Each physical node is placed at many points (**virtual nodes**, 150) so load is even. Doing it in app code keeps the interesting logic visible and explainable. *Trade-off*: we manage routing ourselves instead of getting it free from Redis Cluster — accepted, because the assignment is about understanding the mechanism.

Trending — recency-aware ranking (the 5 required points)

1. **How recent searches are tracked**: each row has `recent_score`; on every search we decay the stored score by elapsed time then add the new activity: $\text{recent_score} = \text{recent_score} \cdot e^{(-\lambda \cdot \Delta t)} + \text{delta}$, with $\lambda = \ln 2 / \text{half_life}$ (default 10 min).
2. **How recency affects ranking**: trending score = `count +  $\alpha$  · decayed_recent_score` (α scales recent activity to be comparable to all-time counts), so a recently-hot query gets a temporary boost.
3. **Avoiding permanent over-ranking of a brief spike**: the score *decays exponentially* — once the burst stops, the boost halves every half-life and the query slides back to its count-based position.
4. **Cache update/invalidation when rankings change**: a changed query invalidates its prefixes' cached entries in both modes; short TTL is a backstop. Basic and trending use separate cache keys.
5. **Trade-offs (freshness vs latency vs complexity)**: shorter TTL / aggressive invalidation = fresher but more misses; the decay is computed in SQL at read time, keeping writes simple at a tiny per-read cost. Defaults: 30s TTL, per-flush invalidation, 10-min half-life — all single config values.

Batch writes & failure trade-offs

Writing to Postgres on every search is wasteful. Instead submissions are buffered and **aggregated** (repeated queries summed), then flushed in a single multi-row upsert either every `FLUSH_INTERVAL_SECONDS` or once `BATCH_SIZE` distinct queries are buffered. Result: $\approx 50\times$ fewer DB writes.

What if it crashes before a flush? The buffer is in memory, so we add an optional **write-ahead log**: each submission is appended to a file; on flush we atomically rename the WAL aside, write the batch, then delete it; on startup we replay any leftover WAL. *Honest limits*: we flush to the OS but do not `fsync` per write (covers a process crash, not a power crash — a throughput trade-off), and recovery is **at-least-once** (a crash in the tiny window after commit but before deleting the WAL replays that batch — slight over-count). With the WAL disabled the writer is pure in-memory: faster, but a crash loses the un-flushed buffer.

5. Performance report

Measured locally (12-core Linux, Docker Postgres + 3 Docker Redis nodes, 120k-query dataset). Reproducible via `scripts/benchmark.py`, `batch_demo.py`, `cache_demo.py`.

5.1 Suggestion latency & cache hit rate

Workload: 247 distinct prefixes; 6,000 warm requests at concurrency 12.

Metric	Cold (cache empty → DB)	Warm (cache populated)
client p50	22.1 ms	10.7 ms
client p95	34.6 ms	15.0 ms
client p99	38.4 ms	17.8 ms
throughput	—	~1,100 req/s

Server-side suggestion latency (cache + DB, excluding HTTP overhead), from `/metrics`: **p50 0.76 ms · p95 2.65 ms · p99 14.98 ms**. **Cache hit rate 96.0%** (6,000 / 6,247). **DB reads: 247** — one per distinct prefix; every repeat served from cache (≈25× fewer DB reads).

5.2 Write reduction via batching

2,000 `POST /search` across 20 distinct queries:

Search submissions	DB row writes	Batch flushes	Reduction
2,000	40	2	98% fewer (≈50×)

5.3 Consistent-hashing behaviour

20,000 keys across 3 nodes (150 virtual nodes each):

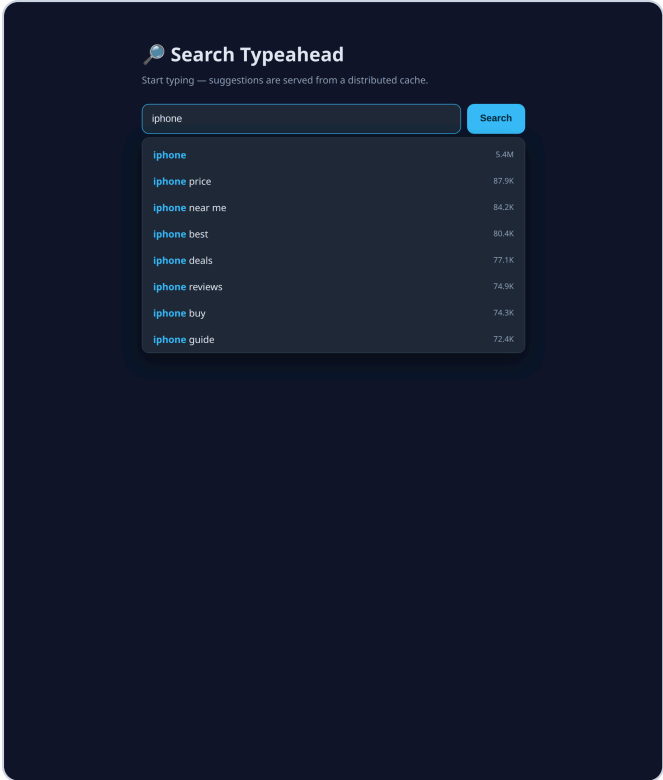
Node	Keys	Skew vs even split
localhost:6391	6,396	−4.1%
localhost:6392	6,283	−5.8%
localhost:6393	7,321	+9.8%

Removing a node remapped **36.6%** of keys (≈ the 33.3% target), versus ~100% for `hash % N`.

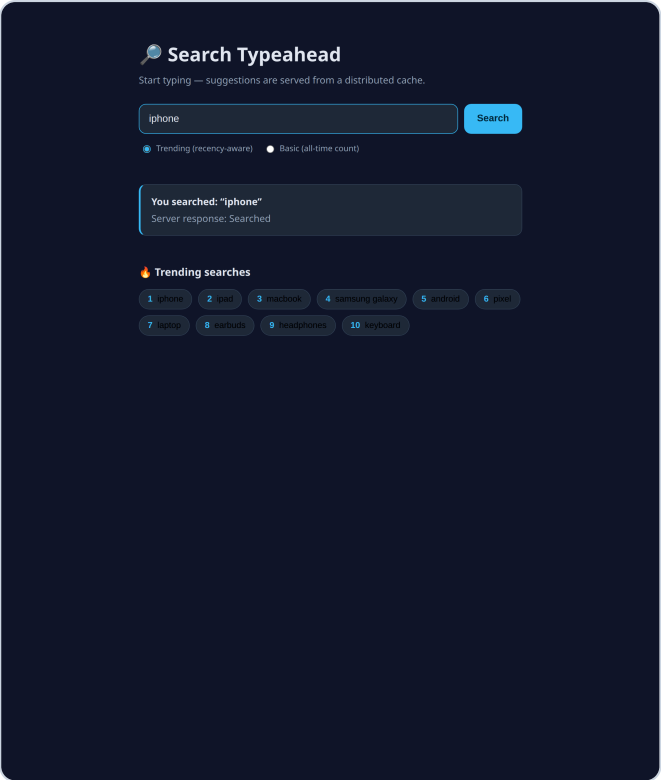
5.4 Crash recovery

30 buffered searches were submitted, the server was `kill -9` 'd (no clean flush), and on restart the batch writer logged `recovered 30 searches from WAL` and the count appeared in Postgres — confirming buffered writes survive a process crash.

5.5 Screenshots



Live typeahead dropdown (prefix highlighted, counts shown)



Search submission + dummy response + trending

Appendix A — Requirements coverage

Rubric component (100 marks)	Status
Basic Implementation (60): dataset ingestion, search UI, suggestions API, search API, query-count updates, distributed cache w/ consistent hashing	Done ✓
Trending Searches (20): basic + recency-aware ranking, scoring/windowing explained, difference demonstrated	Done ✓
Batch Writes (20): buffer + aggregate + flush, write-reduction evidence, failure trade-offs	Done ✓

Appendix B — How to run

```
cd search-typeahead
docker compose up -d                # Postgres + 3 Redis nodes
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
python scripts/generate_dataset.py && python scripts/load_dataset.py
uvicorn app.main:app                # UI: http://localhost:8000 · API docs: /docs
```

Full source, comments, and the markdown versions of these sections are in the repository under `app/`, `docs/`, and `README.md`.